

‘Creación de una Base de Datos’

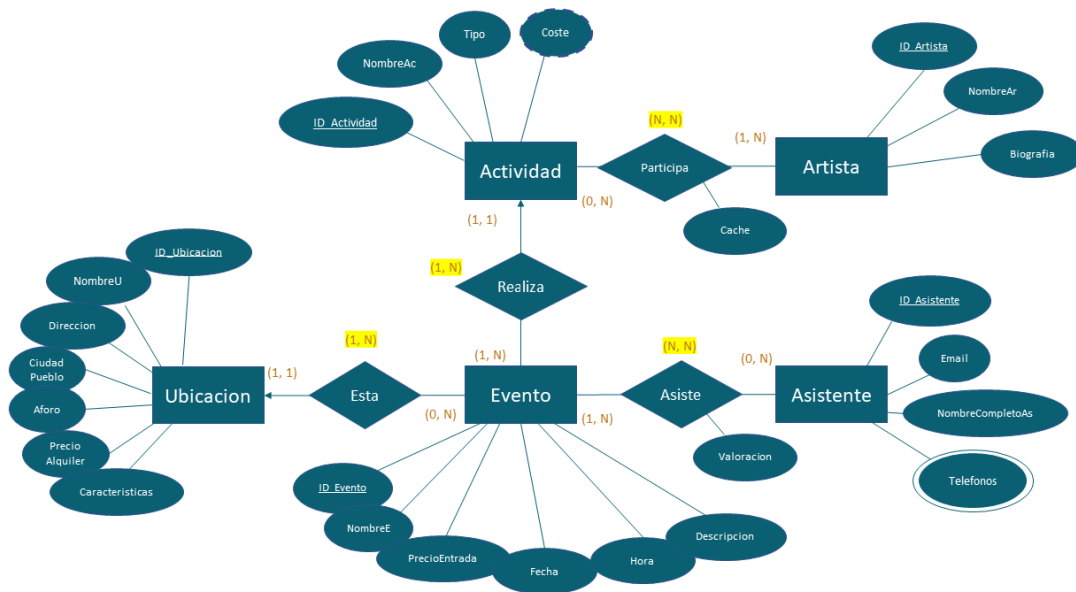
Proyecto Final

ArteVida Cultural

Alumno: David Carrera Pintor

Profesor: Isabel Riomoros

1. Diseño E-R



2. Modelo Relacional

El siguiente paso, es crear las tablas del anterior esquema Entidad-Relación.

En primer lugar, se define la entidad **Actividad**. Para garantizar la atomicidad de la clave primaria, se opta por un campo autoincrementable denominado ID_Actividad. Se incluyen también los demás atributos, NombreAc (cadena de caracteres), Tipo (valor enumerado entre 'Concierto', 'Exposición', 'Teatro' o 'Conferencia') y Coste. Es importante señalar que Coste es un atributo derivado, ya que se calculará posteriormente como la suma de los cachés de los artistas que participan en dicha actividad, por eso no se incluye en la tabla.

La entidad **UBICACION** también tendrá como clave primaria un autoincrementable ID_Ubicacion. Se definen los atributos NombreU (cadena), Direccion (cadena), CiudadPueblo (cadena), Aforo (entero positivo), PrecioAlquiler (con dos decimales y mayor o igual a 0) y Caracteristicas (texto libre), proporcionando toda la información necesaria sobre el lugar en el que se pueden celebrar eventos.

Al igual que las anteriores, la entidad **EVENTO** tiene un campo autoincrementable denominado ID_Evento, que actúa también como clave primaria. Se incluyen los atributos NombreE (cadena), PrecioEntrada (con dos decimales y mayor o igual a 0), Fecha (fecha), Hora (hora) y Descripcion (texto libre). Además, se agregan dos claves foráneas: **ID_Actividad**, que enlaza el evento con la **Actividad** en la que estará, e **ID_Ubicacion**, enlazándolo con una **Ubicación** específica.

Las relaciones Esta y Realiza son tipo uno a varios, por lo que no crean tabla, simplemente se coloca su clave primaria como clave foránea en EVENTO para enlazarlo con Ubicacion y Actividad que son las entidades del lado de uno, al poder estar los eventos solo en una ubicación y realizar una actividad.

En la entidad **ASISTENTE** se crea la tabla con el identificador único autoincrementable ID_Asistente, que representa la clave primaria. Se incorporan los atributos NombreCompletoAs (cadena), Email (cadena con formato válido de correo, pej.: david@gmail.com) y Telefonos. Este último, al poder tener varios valores para cada asistente (multivalorado), crea una nueva tabla, **Telefonos**, con el ID_Asistente como

clave foránea para enlazarlo con el y cada teléfono se representa como una cadena numérica de 9 dígitos. Esto servirá para añadir a cada persona y su información que se registra para asistir a los eventos.

Para la entidad **ARTISTA** se diseña la tabla con un campo autoincrementable denominado ID_Artista, el cual funciona como clave primaria. Se añaden los atributos NombreAr (cadena) y Biografia (texto libre), permitiendo almacenar de manera completa la información referente al artista.

Por otro lado, un artista puede participar en varios eventos, y en un evento pueden participar varios artistas (N:N), por lo que se debe crear una nueva tabla para esta relación, **Participa**. Esta tendrá de claves foráneas el ID_Artista y ID_Actividad, junto con el cache (con dos decimales y mayor o igual a 0) que cobra el artista que cobra por esa actividad ya que es distinta para cada una.

Lo mismo pasa con la relación entre Asistente y Evento, la relación sería varios a varios, por lo que se crea la tabla **Asiste**, con las claves foráneas ID_Asistente e ID_Evento para enlazar estos y el atributo Valoración, que es un valor entero entre 0 y 5 inclusive.

- ACTIVIDAD (ID_Actividad, NombreAc, Tipo)
- UBICACION (ID_Ubicacion, NombreU, Direccion, CiudadPueblo, Aforo, PrecioAlquiler, Caracteristicas)
- EVENTO (ID_Evento, NombreE, PrecioEntrada, Fecha, Hora, Descripcion, ID_Actividad (FK), ID_Ubicacion (FK))
- ASISTENTE (ID_Asistente, NombreCompletoAs, Email)
- ARTISTA (ID_Artista, NombreAr, Biografia)

- Telefonos(ID_Asistente, telefono)
- Participa (ID_Artista, ID_Actividad, Cache)
- Asiste (ID_Asistente, ID_Evento, Valoracion)

3. Script Completo SQL

```
/* -----  
Nombre del autor: David Carrera Pintor  
Nombre de la base de datos: ArteVida_Cultural  
-----*/  
  
DROP DATABASE IF EXISTS ArteVida_Cultural;  
CREATE DATABASE ArteVida_Cultural;  
USE ArteVida_Cultural;  
  
/* -----  
Definición de la estructura de la base de datos  
-----*/  
  
-- Tabla UBICACION  
CREATE TABLE Ubicacion (  
    ID_Ubicacion INT AUTO_INCREMENT PRIMARY KEY,  
    NombreU VARCHAR(100) NOT NULL,  
    Direccion VARCHAR(255) NOT NULL,  
    CiudadPueblo VARCHAR(100) NOT NULL,  
    Aforo INT NOT NULL CHECK (Aforo > 0),  
    PrecioAlquiler DECIMAL(10,2) NOT NULL CHECK (PrecioAlquiler >= 0),  
    Caracteristicas TEXT  
);  
  
-- Tabla ACTIVIDAD  
CREATE TABLE Actividad (  
    ID_Actividad INT AUTO_INCREMENT PRIMARY KEY,  
    NombreAc VARCHAR(100) NOT NULL,  
    Tipo ENUM('Concierto', 'Exposición', 'Teatro', 'Conferencia') NOT NULL  
);
```

-- Tabla EVENTO

```
CREATE TABLE Evento (  
    ID_Evento INT AUTO_INCREMENT PRIMARY KEY,  
    NombreE VARCHAR(150) NOT NULL,  
    PrecioEntrada DECIMAL(10,2) NOT NULL CHECK (PrecioEntrada >= 0),  
    Fecha DATE NOT NULL,  
    Hora TIME NOT NULL,  
    Descripcion TEXT,  
    ID_Actividad INT NOT NULL,  
    ID_Ubicacion INT NOT NULL,  
    FOREIGN KEY (ID_Actividad) REFERENCES Actividad(ID_Actividad) ON  
DELETE CASCADE,  
    FOREIGN KEY (ID_Ubicacion) REFERENCES Ubicacion(ID_Ubicacion) ON  
DELETE CASCADE  
);
```

-- Tabla ARTISTA

```
CREATE TABLE Artista (  
    ID_Artista INT AUTO_INCREMENT PRIMARY KEY,  
    NombreAr VARCHAR(100) NOT NULL,  
    Biografia TEXT  
);
```

-- Tabla ASISTENTE

```
CREATE TABLE Asistente (  
    ID_Asistente INT AUTO_INCREMENT PRIMARY KEY,  
    NombreCompletoAs VARCHAR(150) NOT NULL,  
    Email VARCHAR(100) NOT NULL UNIQUE check (Email REGEXP  
'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')  
);
```

-- Tabla Telefonos

```
CREATE TABLE Telefonos (  
    ID_Asistente INT,  
    Telefono varchar(9) not null check(Telefono REGEXP '^[0-9]{9}$'),  
    PRIMARY KEY (ID_Asistente, Telefono),  
    FOREIGN KEY (ID_Asistente) REFERENCES Asistente(ID_Asistente) ON  
DELETE CASCADE  
);
```

-- Tabla Participa (modificada)

```
CREATE TABLE Participa (  
    ID_Artista INT,  
    ID_Actividad INT,  
    Cache DECIMAL(10,2) NOT NULL CHECK (Cache >= 0),  
    PRIMARY KEY (ID_Artista, ID_Actividad),  
    FOREIGN KEY (ID_Artista) REFERENCES Artista(ID_Artista) ON DELETE  
CASCADE,  
    FOREIGN KEY (ID_Actividad) REFERENCES Actividad(ID_Actividad) ON  
DELETE CASCADE  
);
```

-- Tabla Asiste

```
CREATE TABLE Asiste (  
    ID_Asistente INT,  
    ID_Evento INT,  
    Valoracion TINYINT CHECK (Valoracion BETWEEN 0 AND 5),  
    PRIMARY KEY (ID_Asistente, ID_Evento),  
    FOREIGN KEY (ID_Asistente) REFERENCES Asistente(ID_Asistente) ON  
DELETE CASCADE,  
    FOREIGN KEY (ID_Evento) REFERENCES Evento(ID_Evento) ON DELETE  
CASCADE  
);
```

```
/*-----  
Trigger  
Inserción de datos  
-----*/
```

```
-- Trigger combinado en la tabla Asiste para evitar duplicados y sobrepasar el aforo de la ubicación
```

```
DELIMITER $$
```

```
CREATE TRIGGER before_insert_asiste
```

```
BEFORE INSERT ON Asiste
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    DECLARE total_asistentes INT;
```

```
    DECLARE aforo_maximo INT;
```

```
-- Verificar que el asistente no esté ya registrado en el evento
```

```
IF EXISTS (
```

```
    SELECT 1 FROM Asiste
```

```
    WHERE ID_Asistente = NEW.ID_Asistente
```

```
    AND ID_Evento = NEW.ID_Evento
```

```
) THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El asistente ya está registrado en este evento.';
```

```
END IF;
```

```
-- Verificar que no se supere el aforo de la ubicación del evento
```

```
SELECT COUNT(*) INTO total_asistentes
```

```
FROM Asiste
```

```
WHERE ID_Evento = NEW.ID_Evento;
```

```

SELECT Aforo INTO aforo_maximo
FROM Ubicacion
WHERE ID_Ubicacion = (SELECT ID_Ubicacion FROM Evento WHERE
ID_Evento = NEW.ID_Evento);

IF total_asistentes >= aforo_maximo THEN

    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No se pueden añadir más
asistentes, aforo máximo alcanzado';

END IF;
END$$
DELIMITER ;

```

```

-- Trigger para validar el formato del email al insertar un nuevo asistente
DELIMITER $$
CREATE TRIGGER before_insert_asistente_email
BEFORE INSERT ON Asistente
FOR EACH ROW
BEGIN
    IF NEW.Email NOT REGEXP
'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$' THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El formato del correo
electrónico no es válido.';

    END IF;
END$$
DELIMITER ;

```

```

-- Trigger para calcular automáticamente el precio total de un evento al insertarlo.
-- El precio total será la suma los cachés de artistas de la actividad y con beneficio del
50%.
DELIMITER $$
CREATE TRIGGER before_insert_evento
BEFORE INSERT ON Evento

```


FOR EACH ROW

BEGIN

DECLARE total DECIMAL(10,2);

SELECT COALESCE(SUM(Cache), 0) INTO total

FROM Participa

WHERE ID_Actividad = NEW.ID_Actividad;

SET NEW.PrecioEntrada = total * 1.5;

END\$\$

DELIMITER ;

-- Inserción de datos en las tablas

-- Ubicaciones

INSERT INTO Ubicacion (NombreU, Direccion, CiudadPueblo, Aforo, PrecioAlquiler, Caracteristicas)

VALUES

('Teatro Nacional', 'Av. Central 123', 'Madrid', 500, 2000.00, 'Teatro con excelente acústica y gran historia'),

('Centro Cultural La Plazuela', 'Calle 4, Zona 5', 'Barcelona', 300, 1500.00, 'Espacio para exposiciones y eventos'),

('Auditorio de Sevilla', 'Calle Real 45', 'Sevilla', 400, 1800.00, 'Auditorio moderno'),

('Sala Modernista', 'Av. del Arte 10', 'Valencia', 250, 1200.00, 'Sala para eventos culturales y conferencias');

-- Actividades

INSERT INTO Actividad (NombreAc, Tipo)

VALUES

('Concierto de Rock', 'Concierto'),

('Exposición de Arte Contemporáneo', 'Exposición'),

('Obra de Teatro: Hamlet', 'Teatro'),

('Conferencia sobre Inteligencia Artificial', 'Conferencia'),

('Concierto de Jazz', 'Concierto');

-- Artistas

INSERT INTO Artista (NombreAr, Biografia)

VALUES

('Juan Pérez', 'Artista versátil, reconocido por sus aportes al mundo musical y teatral'),

('Ana Gómez', 'Cantante y compositora, con una carrera llena de éxitos internacionales'),

('Luis Martínez', 'Músico y actor, destacado en diversas producciones artísticas'),

('María López', 'Instrumentista y vocalista, con una amplia trayectoria en conciertos de jazz y rock');

-- Participa

-- Relacionando artistas con las actividades según el caché asignado directamente en la tabla Participa

INSERT INTO Participa (ID_Artista, ID_Actividad, Cache)

VALUES

(1, 1, 800.00), -- Juan Pérez en Concierto de Rock

(3, 1, 600.00), -- Luis Martínez en Concierto de Rock

(4, 1, 1000.00), -- María López en Concierto de Rock

(2, 2, 500.00), -- Ana Gómez en Exposición de Arte

(4, 2, 400.00), -- María López en Exposición de Arte

(3, 3, 800.00), -- Luis Martínez en Obra de Teatro

(1, 4, 600.00), -- Juan Pérez en Conferencia de IA

(2, 4, 400.00), -- Ana Gómez en Conferencia de IA

(4, 5, 750.00), -- María López en Concierto de Jazz

(3, 5, 650.00); -- Luis Martínez en Concierto de Jazz

-- Eventos

INSERT INTO Evento (NombreE, PrecioEntrada, Fecha, Hora, Descripcion, ID_Actividad, ID_Ubicacion)

VALUES

('Concierto de Rock en el Teatro Nacional', 0, '2025-05-15', '20:00:00', 'Un concierto espectacular con bandas internacionales', 1, 1),

('Exposición de Arte en La Plazuela', 0, '2025-06-10', '09:00:00', 'Muestra de los mejores artistas contemporáneos', 2, 2),

('Obra de Teatro en el Auditorio de Sevilla', 0, '2025-07-20', '19:30:00', 'Representación teatral de la obra clásica Hamlet', 3, 3),

('Conferencia de IA en Sala Modernista', 0, '2025-08-12', '18:00:00', 'Ponentes expertos en IA y sus aplicaciones actuales', 4, 4),

('Jazz en vivo en el Auditorio de Sevilla', 0, '2025-09-25', '21:00:00', 'Un concierto de jazz con artistas locales e internacionales', 5, 3);

-- Asistentes

INSERT INTO Asistente (NombreCompletoAs, Email)

VALUES

('Carlos Pérez', 'carlos.perez@gmail.com'),

('Lucía Fernández', 'lucia.f@gmail.com'),

('José Martínez', 'jose.martinez@outlook.com'),

('Paula Vera', 'paulaveraronda@gmail.com'),

('Sergio Ruiz', 'sergio.ruiz@correo.com');

-- Teléfonos

INSERT INTO Telefonos (ID_Asistente, Telefono)

VALUES

(1, '612345678'),

(2, '613456789'),

(3, '614567890'),

(4, '615678901'),

(5, '616789012');

-- Asiste

-- Relacionando asistentes con eventos y sus valoraciones

INSERT INTO Asiste (ID_Asistente, ID_Evento, Valoracion)

VALUES

(1, 1, 5), -- Carlos Pérez en Concierto de Rock
(2, 1, 4), -- Lucía Fernández en Concierto de Rock
(3, 1, 5), -- José Martínez en Concierto de Rock
(3, 3, 3), -- José Martínez en Obra de Teatro
(4, 4, 5), -- Paula Vera en Conferencia de IA
(5, 5, 4), -- Sergio Ruiz en Jazz en vivo
(4, 5, 5), -- Paula Vera en Jazz en vivo
(2, 5, 5); -- Lucía Fernández en Jazz en vivo

/*-----

Consultas, modificaciones, borrados y vistas con enunciado

-----*/

-- Vista 1: Detalle de eventos con información

CREATE VIEW Vista_Evento_Detalle AS

SELECT

E.ID_Evento,

E.NombreE,

E.Fecha,

E.ID_Actividad,

A.NombreAc,

A.Tipo,

U.CiudadPueblo

FROM Evento E

JOIN Actividad A ON E.ID_Actividad = A.ID_Actividad

JOIN Ubicacion U ON E.ID_Ubicacion = U.ID_Ubicacion;

-- Vista 2: Caché total por actividad

CREATE VIEW Vista_CachePorActividad AS

```

SELECT
    Ac.ID_Actividad,
    Ac.NombreAc,
    Ac.Tipo,
    SUM(P.Cache) AS CacheTotal
FROM Actividad Ac
JOIN Participa P ON Ac.ID_Actividad = P.ID_Actividad
GROUP BY Ac.ID_Actividad, Ac.NombreAc, Ac.Tipo;

```

-- Vista 3: Eventos con número de asistentes y porcentaje de ocupación

```

CREATE VIEW Vista_EventoOcupacion AS
SELECT
    E.ID_Evento,
    E.NombreE,
    E.Fecha,
    U.NombreU AS Ubicacion,
    COUNT(AE.ID_Asistente) AS NumeroDeAsistentes,
    ROUND(COUNT(AE.ID_Asistente) * 100.0 / U.Aforo, 2) AS PorcentajeOcupacion
FROM Evento E
JOIN Asiste AE ON E.ID_Evento = AE.ID_Evento
JOIN Ubicacion U ON E.ID_Ubicacion = U.ID_Ubicacion
GROUP BY E.ID_Evento, E.NombreE, E.Fecha, U.NombreU, U.Aforo;

```

-- Consulta 1. Obtener todos los eventos con su nombre, fecha, ubicación y precio.

```

SELECT E.NombreE, E.Fecha, U.NombreU AS Ubicacion, E.PrecioEntrada
FROM Evento E
JOIN Ubicacion U ON E.ID_Ubicacion = U.ID_Ubicacion;

```

-- Consulta 2. Contar el número total de asistentes por evento y mostrar porcentaje de ocupación.

```
SELECT
    NombreE,
    NumeroDeAsistentes,
    PorcentajeOcupacion
FROM Vista_EventoOcupacion;
```

-- Consulta 3. Obtener los artistas que participan en actividades cuyo coste total sea mayor que el coste promedio de todos los artistas en la base de datos.

```
SELECT DISTINCT Ar.NombreAr
FROM Artista Ar
JOIN Participa P ON Ar.ID_Artista = P.ID_Artista
JOIN Vista_CachePorActividad VCA ON P.ID_Actividad = VCA.ID_Actividad
WHERE VCA.CacheTotal > (
    SELECT AVG(CacheTotal)
    FROM Vista_CachePorActividad
);
```

-- Consulta 4. Obtener todos los eventos de tipo Concierto que tengan lugar en Madrid y con un aforo superior a 400.

```
SELECT VED.NombreE, VED.Fecha, VED.CiudadPueblo, U.Aforo
FROM Vista_Evento_Detalle VED
JOIN Ubicacion U ON VED.CiudadPueblo = U.CiudadPueblo
WHERE VED.Tipo = 'Concierto'
    AND VED.CiudadPueblo = 'Madrid'
    AND U.Aforo > 400;
```

-- Consulta 5. Obtener el evento con el mayor precio de entrada y la ciudad en la que se celebra.

```
SELECT VED.NombreE, E.PrecioEntrada, VED.CiudadPueblo
```

```

FROM Evento E
JOIN Vista_Evento_Detalle VED ON E.ID_Evento = VED.ID_Evento
WHERE E.PrecioEntrada = (SELECT MAX(PrecioEntrada) FROM Evento);

```

-- 6. Obtener los artistas, las actividades en las que participan, y el coste asociado de cada artista por actividad.

```

SELECT Ar.NombreAr, Ac.NombreAc AS Actividad, P.Cache
FROM Artista Ar
JOIN Participa P ON Ar.ID_Artista = P.ID_Artista
JOIN Actividad Ac ON P.ID_Actividad = Ac.ID_Actividad;

```

-- 7. Obtener los eventos que tienen un precio de entrada mayor que el promedio de precios de todos los eventos de un tipo específico, Concierto en este caso.

```

SELECT E.NombreE, E.PrecioEntrada, A.Tipo
FROM Evento E
LEFT JOIN Actividad A ON E.ID_Actividad = A.ID_Actividad
LEFT JOIN (
    SELECT AVG(E2.PrecioEntrada) AS Promedio
    FROM Evento E2
    JOIN Actividad A2 ON E2.ID_Actividad = A2.ID_Actividad
    WHERE A2.Tipo = 'Concierto'
) PromedioPrecios ON 1 = 1
WHERE A.Tipo = 'Concierto'
AND E.PrecioEntrada > PromedioPrecios.Promedio;

```

-- 8. Obtener los artistas que participan en actividades que tienen un coste total de caché mayor

-- que el promedio de los costes de actividades del mismo tipo.

```

SELECT DISTINCT Ar.NombreAr AS NombreArtista
FROM Artista Ar
JOIN Participa P ON Ar.ID_Artista = P.ID_Artista

```

```

JOIN Vista_CachePorActividad VCA ON P.ID_Actividad = VCA.ID_Actividad
WHERE VCA.CacheTotal > (
    SELECT AVG(CacheTotal)
    FROM Vista_CachePorActividad V2
    WHERE V2.Tipo = VCA.Tipo
);

```

-- Consulta 9: Obtiene por cada ciudad, el evento con mayor asistencia y media de valoración mayor a 4, su tipo de actividad,

-- el porcentaje de ocupación y los artistas participantes

```

SELECT
    ved.CiudadPueblo,
    ved.NombreE AS Evento,
    ved.Tipo AS TipoActividad,
    veo.NumeroDeAsistentes,
    veo.PorcentajeOcupacion,
    (SELECT GROUP_CONCAT(DISTINCT a.NombreAr SEPARATOR ', ')
    FROM Participa p
    JOIN Artista a ON p.ID_Artista = a.ID_Artista
    WHERE p.ID_Actividad = ved.ID_Actividad
    ) AS Artistas
FROM Vista_Evento_Detalle ved
JOIN Vista_EventoOcupacion veo ON ved.ID_Evento = veo.ID_Evento
WHERE ved.ID_Evento = (
    SELECT sub.ID_Evento
    FROM Vista_Evento_Detalle sub
    JOIN Vista_EventoOcupacion subve ON sub.ID_Evento = subve.ID_Evento
    WHERE sub.CiudadPueblo = ved.CiudadPueblo
    ORDER BY subve.NumeroDeAsistentes DESC
    LIMIT 1
)

```



```

AND (
    SELECT AVG(Valoracion)
    FROM Asiste
    WHERE ID_Evento = ved.ID_Evento
) > 4
ORDER BY ved.CiudadPueblo;

```

-- 10. Obtener los eventos en cada ciudad que tienen el mayor promedio de valoración,
 -- siempre que tengan al menos 3 asistentes, junto con su nombre, fecha, precio de
 entrada y número de asistentes.

```

SELECT
    U.CiudadPueblo,
    E.NombreE AS Evento,
    E.Fecha,
    E.PrecioEntrada,
    COUNT(A.ID_Asistente) AS NumeroDeAsistentes,
    AVG(ASIS.Valoracion) AS PromedioValoracion
FROM Evento E
JOIN Ubicacion U ON E.ID_Ubicacion = U.ID_Ubicacion
JOIN Asiste ASIS ON E.ID_Evento = ASIS.ID_Evento
JOIN Asistente A ON ASIS.ID_Asistente = A.ID_Asistente
GROUP BY U.CiudadPueblo, E.ID_Evento
HAVING COUNT(A.ID_Asistente) >= 3
AND AVG(ASIS.Valoracion) >= (
    SELECT FLOOR(MAX(PromedioValoracion))
    FROM (
        SELECT E2.ID_Evento, AVG(ASIS2.Valoracion) AS PromedioValoracion
        FROM Evento E2
        JOIN Asiste ASIS2 ON E2.ID_Evento = ASIS2.ID_Evento
        JOIN Ubicacion U2 ON E2.ID_Ubicacion = U2.ID_Ubicacion
        WHERE U2.CiudadPueblo = U.CiudadPueblo
    )
)

```

```
GROUP BY E2.ID_Evento
HAVING COUNT(ASIS2.ID_Asistente) >= 3
) AS Subquery
)
ORDER BY U.CiudadPueblo;
```